Kodewala Academy(1st June, 2026) - Java Backend Test # 9

Daksh Arya
Sept. 11, 2026, 12:22 p.m.

You've survived the test. The results will arrive soon. 😎

Your score: 88% 26.25/30
Duration: 0:21:42

1. 1 / 1 point

```
String s1 = new String("Kodewala");
String s2 = "Kodewala";
String s3 = s1.intern();
System.out.println(s1 == s2);
System.out.println(s2 == s3);
```

What is the output?

Your Answer: ✓ Correct

- ○ true true
- ✓ ● false true
- ○ true false
- ○ false false

2. 1 / 1 point

```
String s = "Java";
s.concat("8");
s.replace("Java", "Advanced");
System.out.println(s);
```

What will be printed?

Your Answer: ✓ Correct

- ○ Advanced8
- ○ Java8
- ○ Advanced
- ✓ ● Java

3. 1 / 1 point

```
String a = "Kode" + "wala";
String b = "Kodewala";
System.out.println(a == b);
```

Why?

Your Answer: ✓ Correct

- ○ Both are created using new
- ✓ ● Compile-time constants are resolved and placed in the String Pool
- ○ == compares String contents
- ○ VM always interns every String

4. 1 / 1 point

```
class Employee {
    int id;
    Employee(int id) {
        this.id = id;
    }
    public int hashCode() {
        return id;
    }
    public boolean equals(Object o) {
        return o instanceof Employee && ((Employee)o).id == id;
    }
}
```

If you use :

```
Employee e = new Employee(101);
Map<Employee, String> map = new HashMap<>();
map.put(e, "John");
e.id = 102;
System.out.println(map.get(e));
```

Your Answer: ✓ Correct

- ○ John
- ✓ ● null
- ○ Compilation error
- ○ 102

5. -0.25 / 1 point

Two different keys produce the same hash code but equals() returns false.

What does HashMap do?

Your Answer: ✗ Incorrect

- ○ Rejects the second key
- ✗ ● Replaces the first key
- ✓ ○ Stores both entries in the same bucket
- ○ Throws Exception

6. 1 / 1 point

A class overrides equals() but does not override hashCode().

What problem can occur when objects of this class are used as HashMap keys?

Your Answer: ✓ Correct

- ○ Compilation failure
- ✓ ● Equal objects may be stored/retrieved incorrectly
- ○ HashMap automatically generates hashCode
- ○ Nothing; equals() is sufficient

7. 1 / 1 point

```
Set<String> set = new TreeSet<>( (a, b) -> a.length() - b.length()
);
set.add("Java");
set.add("Spring");
set.add("API");
System.out.println(set);
```

What is the key issue?

Your Answer: ✓ Correct

- ○ All values are stored
- ○ Strings are sorted alphabetically
- ✓ ● Strings having equal length may be considered duplicates
- ○ Runtime exception occurs

8. 1 / 1 point

You have an Employee class where the default ordering should be based on employee ID, but different screens need sorting by salary and name.

What is the best design?

Your Answer: ✓ Correct

- ○ Implement multiple Comparable interfaces
- ✓ ● Use Comparable for ID and separate Comparators for salary/name
- ○ Use only Comparator
- ○ Override hashCode()

9. 1 / 1 point

Which collection is commonly considered fail-safe because its iterator works over a snapshot?

Your Answer: ✓ Correct

- ○ ArrayList
- ○ HashSet
- ✓ ● CopyOnWriteArrayList
- ○ TreeSet

10. 1 / 1 point

```
static void m1() {
    throw new RuntimeException("Error");
}
static void m2() {
    m1();
}
static void m3() {
    try {
        m2();
    } catch(RuntimeException e) {
        System.out.println("Handled");
    }
}
```

What happens when m3() is called?

Your Answer: ✓ Correct

- ○ Compilation error
- ○ JVM terminates immediately
- ✓ ● Handled is printed
- ○ finally is mandatory

11. 1 / 1 point

```
static int test() {
    try {
        return 10;
    } finally {
        return 20;
    }
}
```

What does test() return?

Your Answer: ✓ Correct

- ○ 10
- ✓ ● 20
- ○ Compilation error
- ○ Depends on JVM

12. 1 / 1 point

You want an exception indicating an invalid business operation that callers are expected to explicitly handle.

Which is generally more appropriate?

Your Answer: ✓ Correct

- ○ Extend Error
- ○ Extend RuntimeException always
- ○ Extend Throwable directly
- ✓ ● Extend Exception

13. 1 / 1 point

Which interface can be used directly with a lambda?

Your Answer: ✓ Correct

- ○ Runnable
- ○ Consumer
- ○ Function
- ✓ ● All of the options

14. 1 / 1 point

Two interfaces provide the same default method:

```
interface A {
    default void show() {}
}
interface B {
    default void show() {}
}
```

A class implements both.

What must the class do?

Your Answer: ✓ Correct

- ○ Nothing
- ✓ ● Override show() to resolve the conflict
- ○ Mark class abstract
- ○ Use super.show()

15. 1 / 1 point

```
void process()
{
    Employee e = new Employee();
}
```

Where is the Employee object primarily allocated?

Your Answer: ✓ Correct

- ○ Stack
- ✓ ● Heap
- ○ Method Area only
- ○ CPU register only

16. 1 / 1 point

Which scenario can cause a logical memory leak despite garbage collection?

Your Answer: ✓ Correct

- ○ Local variable goes out of scope
- ○ Creating immutable objects
- ○ Calling System.gc()
- ✓ ● Object remains strongly referenced by a long-lived static collection

17. 1 / 1 point

What is the key characteristic of WeakHashMap?

Your Answer: ✓ Correct

- ○ Values are weakly referenced
- ✓ ● Keys may be garbage-collected when no strong references remain
- ○ It never allows garbage collection
- ○ It is always thread-safe

18. -0.25 / 1 point

```
String s1 = new String("Java");
String s2 = new String("Java");
System.out.println(s1 == s2);
System.out.println(s1.equals(s2));
```

Your Answer: ✗ Incorrect

- ○ true true
- ○ false false
- ✓ ○ false true
- ✗ ● true false

19. 1 / 1 point

An application has 1 million customer records. The requirements are:

- Customer ID must be unique
- Very frequent lookup by Customer ID
- Frequent insert/update operations
- Ordering is not required
- Multiple threads may access the collection concurrently

Which is the most appropriate choice?

Your Answer: ✓ Correct

- ✓ ● ConcurrentHashMap<CustomerId, Customer>
- ○ LinkedList
- ○ TreeMap
- ○ ArrayList

20. 1 / 1 point

```
class Employee {
    int id;
    String name;
    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }
    @Override
    public boolean equals(Object obj) {
        Employee e = (Employee) obj;
        return this.id == e.id;
    }
}
```

Then:

```
Map<Employee, String> map = new HashMap<>();
Employee e1 = new Employee(101, "Rahul");
Employee e2 = new Employee(101, "Amit");
map.put(e1, "Developer");
map.put(e2, "Tester");
```

System.out.println(map.size());

What is the most appropriate answer?

Your Answer: ✓ Correct

- ○ 1 because equals() returns true
- ✓ ● 2 because hashCode() is not overridden, so the objects may go into different buckets
- ○ Compilation error
- ○ 2 because HashMap ignores equals()

21. 1 / 1 point

```
List<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
List<Integer> readOnly = Collections.unmodifiableList(list);
list.add(30);
System.out.println(readOnly);
```

What will be the output?

Your Answer: ✓ Correct

- ○ [10, 20]
- ○ [10, 20, 30]
- ○ Compilation error
- ✓ ● UnsupportedOperationException

22. -0.25 / 1 point

```
List<Integer> list = Arrays.asList(30, 10, 20);
Collections.sort(list);
System.out.println(list);
```

What will happen?

Your Answer: ✗ Incorrect

- ○ [30, 10, 20]
- ✓ ○ [10, 20, 30]
- ○ Compilation error
- ✗ ● UnsupportedOperationException

23. 1 / 1 point

A multi-threaded application has multiple threads reading and updating a shared map.

Which statement is most accurate?

Your Answer: ✓ Correct

- ○ HashMap automatically synchronizes all operations, while ConcurrentHashMap does not
- ✓ ● ConcurrentHashMap is designed for concurrent access, whereas HashMap is not thread-safe
- ○ Both provide the same thread-safety guarantees
- ○ ConcurrentHashMap allows null keys and values, while HashMap does not

24. 1 / 1 point

A HashMap contains n entries. Due to a poorly implemented hashCode(), all keys produce the same hash value.

In a modern Java implementation, assuming the bucket becomes a tree when the relevant threshold is reached, what is the expected lookup complexity for a key in that heavily-collided bucket?

Your Answer: ✓ Correct

- ○ Always O(1)
- ✓ ● O(log n) after treeification
- ○ Always O(n)
- ○ O(n log n)

25. 1 / 1 point

You need to process 1 million records with these operations:

```
map.put(key, value);
map.get(key);
map.remove(key);
```

The application must maintain keys in sorted order after every modification.

Which statement correctly describes the typical complexity?

Your Answer: ✓ Correct

- ○ HashMap: O(log n) for all operations; TreeMap: O(1)
- ✓ ● HashMap: O(1) average; TreeMap: O(log n) for put/get/remove
- ○ Both are O(1) average
- ○ HashMap: O(log n); TreeMap: O(n)

26. 1 / 1 point

```
class Parent {
    static void show() {
        System.out.println("Parent");
    }
}
class Child extends Parent {
    static void show() {
        System.out.println("Child");
    }
}
public class Test {
    public static void main(String[] args) {
        Parent obj = new Child();
        obj.show();
    }
}
```

Your Answer: ✓ Correct

- ✓ ● Parent
- ○ Child
- ○ Compilation error
- ○ Runtime exception

27. 1 / 1 point

```
class Test {
    static void display(String s) {
        System.out.println("String");
    }
    static void display(Object o) {
        System.out.println("Object");
    }
    public static void main(String[] args) {
        display(null);
    }
}
```

Your Answer: ✓ Correct

- ✓ ● Prints String
- ○ Prints Object
- ○ Compilation error
- ○ Runtime exception

28. 1 / 1 point

```
Stream.of(1, 2, 3, 4, 5).filter(n -> {
        System.out.print(n + " ");
        return n > 2;
    }).findFirst().ifPresent(n -> System.out.println(n));
```

Your Answer: ✓ Correct

- ○ 1 2 3 4 5
- ✓ ● 1 2 3 3
- ○ 3 3
- ○ 1 2 3 4 5 5

29. 1 / 1 point

```
Integer a = 127;
Integer b = 127;
Integer c = 128;
Integer d = 128;
System.out.println(a == b);
System.out.println(c == d);
```

Your Answer: ✓ Correct

- ○ true true
- ○ false false
- ✓ ● true false
- ○ false true

30. 1 / 1 point

What are you doing every day to achieve your dream job?

Your Answer: ✓ Correct

- ○ Waiting for the right opportunity
- ○ Practicing only when I feel motivated
- ✓ ● Practicing consistently and improving every day
- ○ Giving up when I face difficult problems

Explanation:

Practice today. Confidence tomorrow. Job soon!

Logout